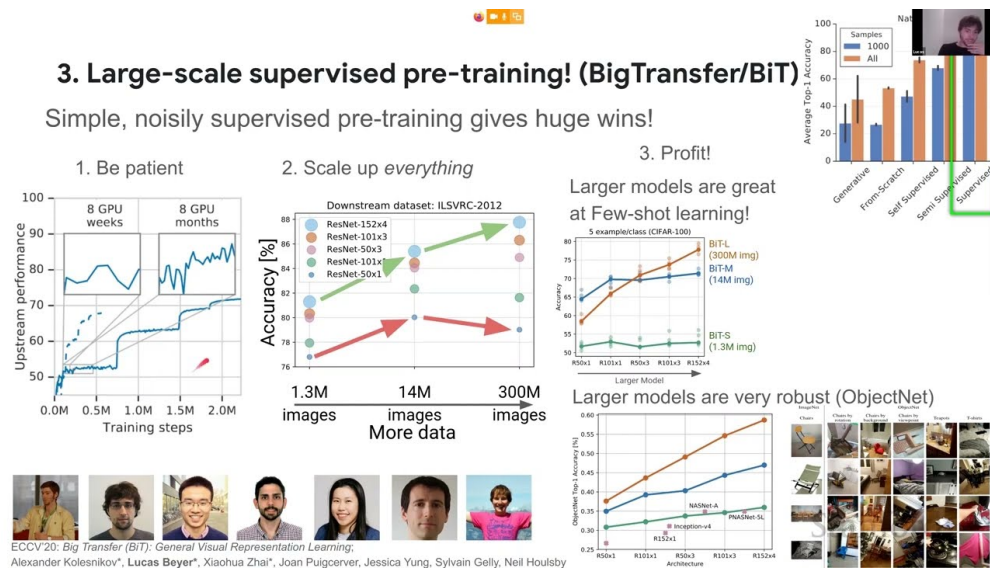


课程笔记

[CS25] Transformers in Vision —Lucas Beyer, Google Brain

基于公开课程资料整理

2021



视频作者/频道: Stanford CS25

发布日期: 2021

视频时长: 约 60 分钟

项目主页: <https://github.com/hqh1025/ai-course-notes>

目录

1 引言：通用视觉表征的目标	2
1.1 VTAB 基准测试	2
1.2 本章小结	2
2 规模化预训练：从 BiT 到 ViT 的铺垫	2
2.1 自监督 vs. 半监督 vs. 全监督	2
2.2 Big Transfer (BiT) 的关键发现	3
2.3 本章小结	3
3 Vision Transformer (ViT)	3
3.1 核心思想：图像 = 序列的 Patch	3
3.2 ViT vs. ResNet：数据量的临界点	4
3.3 位置嵌入的自学习	4
3.4 本章小结	4
4 ViT 的计算效率与进一步探索	4
4.1 自注意力的计算复杂度	4
4.2 ViT 变体与后续改进	5
4.3 本章小结	5
5 总结与延伸	5
5.1 核心要点回顾	5
5.2 拓展阅读	5

1 引言：通用视觉表征的目标

Lucas Beyer 来自 Google Brain，其核心研究目标是寻找**通用视觉表征**（General Visual Representation）。所谓通用视觉表征，是指一种能够在各种视觉任务上快速适应的特征提取能力——类似于人类视觉系统在仅看到少量示例后便能理解新类别的能力。

什么是通用视觉表征？

通用视觉表征指的是：预训练一个模型，使其学到的特征在**广泛的下游视觉任务**上都能快速适应。评测时不仅包括自然图像分类，还包括卫星图像、计数、距离估计等多样化任务。核心思想是 few-shot 学习能力——看少量样本即可泛化。

Lucas 以一系列生动的 few-shot 分类示例开场：花卉分类（人类熟悉的领域）、卫星图像分类（人类较少接触的领域）、以及抽象形状计数（需要高级推理的领域）。这些示例直观地展示了人类视觉系统的泛化能力。

1.1 VTAB 基准测试

为了量化衡量通用视觉表征的质量，Lucas 介绍了 **Visual Task Adaptation Benchmark (VTAB)**：

- 包含 19 个多样化的视觉任务
- 覆盖自然图像、专业图像（卫星等）、非分类任务（计数、距离等）
- 评测流程：预训练模型 → 在每个任务上进行少量样本适应 → 计算平均分数

术语说明

- **预训练/上游 (Pre-training/Upstream)**：在大规模数据上训练通用模型
- **迁移/下游 (Transfer/Downstream)**：将预训练模型适配到具体任务
- **微调 (Fine-tuning)**：在下游任务数据上继续训练模型的所有参数

1.2 本章小结

本节明确了研究目标——找到最好的通用视觉表征，并通过 VTAB 基准来衡量进展。

2 规模化预训练：从 BiT 到 ViT 的铺垫

2.1 自监督 vs. 半监督 vs. 全监督

Lucas 介绍了团队在通用视觉表征上的探索历程：

1. **自监督预训练**：在视觉领域效果有限（当时）
2. **半监督训练**：加入少量标签后表征质量显著提升
3. **大规模全监督预训练 (BiT)**：真正的突破——规模化一切

2.2 Big Transfer (BiT) 的关键发现

BiT 的核心洞察

1. **数据和模型必须同时扩大**：仅增加数据而不增加模型容量，效果会饱和；仅增加模型而数据不够，模型也无法受益
2. **耐心训练**：训练初期看起来没有进展，但长时间训练后性能持续提升
3. **规模化带来鲁棒性**：在 ObjectNet 等对抗性数据集上，大规模预训练模型展现出远超小模型的鲁棒性

数据泄露警告

当使用大规模互联网数据进行预训练时，必须确保下游评测数据集中的图像未出现在预训练集中。Lucas 指出，很多经典视觉数据集（如 ImageNet 训练集和 CIFAR-10/100 测试集之间）存在近重复图像。他们使用内部的 near-duplicate 检测工具来消除这种泄露。

2.3 本章小结

BiT 证明了规模化全监督预训练的威力，打破了 ImageNet 上长期停滞的记录。下一步自然的问题是：能否用更好的架构进一步提升？

3 Vision Transformer (ViT)

3.1 核心思想：图像 = 序列的 Patch

ViT 的核心创新极其简洁：

ViT 的架构设计

1. 将图像切分为固定大小的 patch（如 16×16 像素）
2. 每个 patch 通过线性投影映射到嵌入空间（如 768 维）
3. 添加可学习的位置嵌入（position embedding）
4. 添加一个特殊的 [CLS] token
5. 将所有 token 输入标准的 BERT Transformer 编码器
6. [CLS] token 的输出通过 MLP 头进行分类

关键设计选择：

- **非重叠 patch**：最简单的方案，后续工作尝试了重叠卷积等变体
- **线性投影**：将 patch 的像素值展平后做矩阵乘法
- **完全使用 Transformer**：不依赖任何 CNN 组件

3.2 ViT vs. ResNet: 数据量的临界点

ViT 的性能高度依赖预训练数据规模:

- **ImageNet (1.3M 图像)**: ViT 不如 ResNet
- **ImageNet-21k (13M 图像)**: ViT 与 ResNet 持平
- **JFT-300M (3 亿图像)**: ViT 开始超越 ResNet, 且大模型优势更明显

为什么 ViT 需要更多数据?

CNN 具有**归纳偏置** (inductive bias): 局部连接和权重共享天然编码了平移不变性和局部性。Transformer 没有这些先验, 需要从数据中学习这些结构。数据少时, 归纳偏置有利; 数据充足时, 更灵活的架构有更高的上限。

3.3 位置嵌入的自学习

ViT 使用随机初始化的可学习位置嵌入。训练后可视化发现:

- 每个位置嵌入与其空间邻域的嵌入最相似
- 模型自动从数据中恢复了 2D 空间结构
- 显式编码位置 (如 2D 正弦编码) 或相对位置编码均未能超越自由学习的方案

Patch 顺序敏感性

如果在训练后打乱 patch 输入顺序, 性能会严重下降——因为位置嵌入已经学到了特定的空间关系。但如果训练时保持一致的 (非标准) 顺序, 效果与标准顺序完全相同。

3.4 本章小结

ViT 用最简单的方式将 Transformer 应用于视觉领域, 在大数据场景下超越了精心设计的 CNN。这一结果预示着 Transformer 在视觉领域的广阔前景。

4 ViT 的计算效率与进一步探索

4.1 自注意力的计算复杂度

对于图像尺寸 H 的输入和 patch 大小 p :

$$\text{序列长度} = \left(\frac{H}{p}\right)^2$$

$$\text{自注意力复杂度} \propto \left(\frac{H}{p}\right)^4 = O(H^4/p^4)$$

四次方复杂度

自注意力关于图像尺寸是**四次方复杂度**——这在理论上非常可怕。但实际上, 在当前常用的图像分辨率下, 自注意力的计算量仍小于 MLP 层。只有在非常高分辨率时才成为瓶颈。

4.2 ViT 变体与后续改进

Lucas 提到了 ViT 之后的一些重要发展方向：

- **DeiT**：通过数据增强和知识蒸馏，仅用 ImageNet 训练 ViT
- **重叠 patch 嵌入**：用小型卷积替代线性投影
- **混合模型**：CNN 前端 + Transformer 后端
- **MLP-Mixer**：用 MLP 替代自注意力，探索注意力的必要性

4.3 本章小结

尽管自注意力有四次方复杂度，在实际应用中 ViT 仍然高效。后续研究探索了多种改进方向。

5 总结与延伸

5.1 核心要点回顾

1. **通用视觉表征**是计算机视觉的核心追求，VTAB 提供了衡量标准
2. **规模化是关键**：数据、模型、计算、耐心缺一不可（BiT 的教训）
3. **ViT 的简洁之美**：将图像切成 patch，直接送入标准 Transformer
4. **数据量决定架构优势**：数据少时 CNN 的归纳偏置有利，数据多时 Transformer 的灵活性占优
5. **位置嵌入可自学习**：模型能从数据中恢复空间结构

5.2 拓展阅读

- Dosovitskiy et al., “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale,” ICLR 2021
- Kolesnikov et al., “Big Transfer (BiT): General Visual Representation Learning,” ECCV 2020
- Touvron et al., “Training Data-Efficient Image Transformers (DeiT),” ICML 2021
- Zhai et al., “The Visual Task Adaptation Benchmark (VTAB),” 2019